

UD1. Introducción al desarrollo de software

Módulo: Entornos de desarrollo



Unión Europea

Fondo Social Europeo
El FSE invierte en tu futuro

Germán Gascón Grau
g.gascongrau@edu.gva.es

>_

- ¿Qué se necesita para programar una aplicación?
- ¿Qué pasos hay que seguir para programar?
- ¿Quién interviene en el proceso de desarrollo?
- ¿Todas las aplicaciones están programadas con el mismo lenguaje?

1. Introducción
2. El SW del ordenador
3. Lenguajes de programación
4. El proceso de desarrollo
5. Roles que intervienen en el desarrollo

Introducción



El ordenador se compone de:

- **Hardware** (HW). La parte tangible del ordenador (Caja, CPU, RAM, Pantalla, teclado, ratón, etc.)
- **Software** (SW). La parte intangible del ordenador (programas, sistema operativo, drivers, Firmware de la BIOS, etc.)



HARDWARE



SOFTWARE



está
eso sí,

El SW se puede hacer a medida



- 
- A round, golden-brown Spanish omelette (tortilla) served on a white plate with a fork.

1) Polar I

m

$$h = 7 \text{ cm}$$

m

$$b = 10 \text{ cm}$$

$$A = b \cdot h$$

- Solicitar la base (b)
- Solicitar la altura (h)
- Multiplicar la base por la altura y mostrar el resultado de la operación.

“Ahora, el único límite es tu imaginación. Puedes conseguir que un ordenador haga cosas que antes eran imposibles. Es lo más parecido a tener superpoderes”.

Drew Houston, fundador de Dropbox

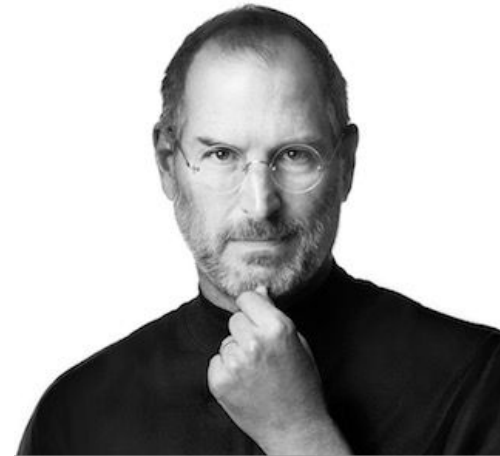


es
cebook

A black and white portrait of Mark Zuckerberg, smiling and wearing a dark jacket. The image is positioned on the left side of the slide, with a large, faint watermark of the same image in the background.

“Creo que todos en este país deberían aprender a programar un ordenador, ya que te enseña a pensar.”

Steve Jobs, fundador de Apple



“La mayoría de los buenos programadores programan no porque esperen que les paguen o recibir elogios, sino porque es divertido programar”

Linus Torvalds, creador de Linux y Git



Lenguajes de programación

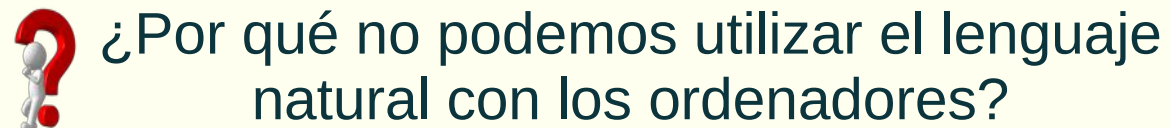


Lenguajes **artificiales** creados por y para programadores.

Permiten **traducir las instrucciones** de un programa a código comprensible para el ordenador.

Son lenguajes que se acercan más al lenguaje humano que al lenguaje máquina (0 y 1).





¿Por qué no podemos utilizar el lenguaje natural con los ordenadores?

- Según el nivel de abstracción
- Según el método de ejecución
- Según el paradigma de programación

Clasificación según el nivel de abstracción



Bajo nivel: cercanos a las particularidades de la máquina (hardware).

- Ventajas: permiten aprovechar al máximo los recursos de la máquina.
- Inconvenientes:
 - Díficiles de programar y mantener
 - Poca o nula portabilidad

Ejemplo: Ensamblador, Lenguaje máquina.

```
mov ax, 3
add ax, 5
mov [0x9000], ax
mov eax, 1
mov ebx, 0
int 0x80
```

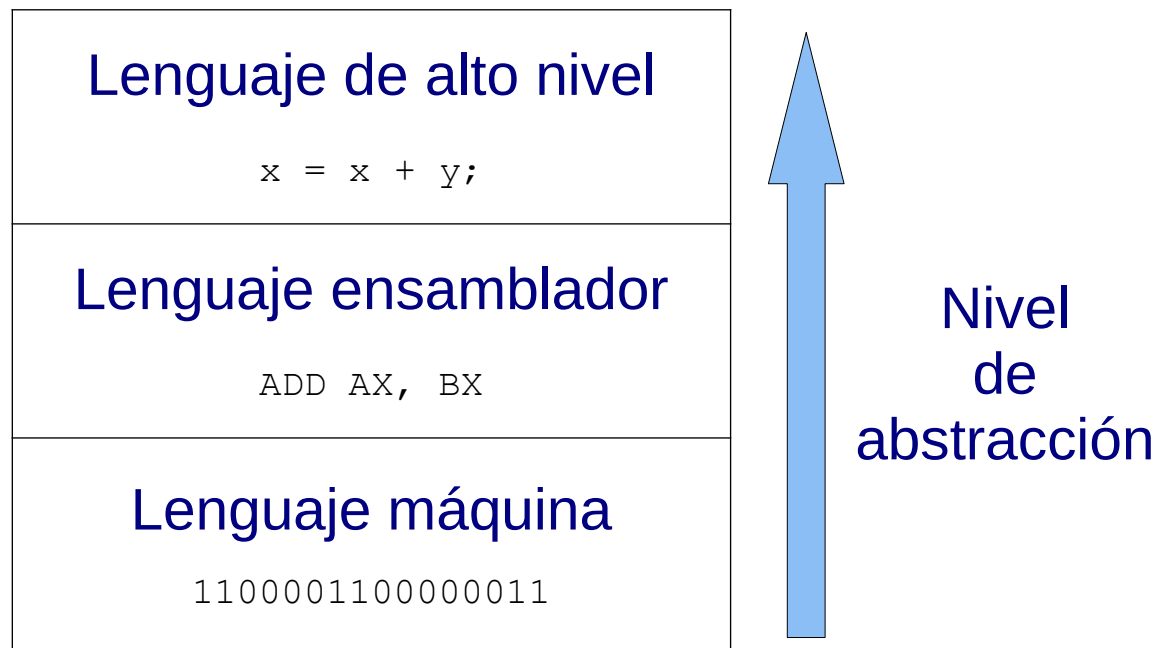
Alto nivel: sintaxis cercana al lenguaje humano.

- Ventajas:
 - Fáciles de aprender y mantener
- Inconvenientes
 - Al ser más genéricos, es difícil que aprovechen al máximo los recursos de la máquina.

Ejemplo: Java, Python, C#.

```
int a = 3;
int b = 5;
int x = 3 + 5;
```





Lenguaje máquina



1100001100000011

Es el único lenguaje que realmente entiende el ordenador.

Características:

- Las instrucciones se expresan en **binario**.
- Se accede a los datos y a las instrucciones mediante **direcciones de memoria y desplazamientos**.
- No existe el concepto de tipos de dato. Los datos se diferencian por **cuantos bytes son leídos y qué uso** se hace de ellos.
- **No aparecen nombres** o identificadores. Sólo datos binarios.
- No pueden incluirse comentarios.
- **Depende de la arquitectura del procesador** del ordenador. Un programa escrito para una arquitectura no se puede ejecutar en otra arquitectura (baja portabilidad).



Lenguaje ensamblador



ADD AX, BX

El lenguaje ensamblador es un primer paso para facilitar la creación de programas.

Al ser humano le es **más fácil recordar palabras** que números.

Está **basado en el conjunto de instrucciones del procesador**.

Cada instrucción del procesador tiene asociado un **mnemónico**. Por ejemplo en la arquitectura x86 la instrucción suma es ADD.

El principal inconveniente es que los programas creados en ensamblador en una arquitectura no sirven para otra.



Lenguaje de alto nivel



$x = x + y;$

Es el siguiente paso en el **acercamiento** de los lenguajes de programación **al lenguaje humano**.

Para ello se basa en algunas palabras del inglés y nomenclatura matemática de forma que las **instrucciones sean claras, concisas y sin ambigüedades**.

El objetivo es llevar la creación de programas al **flujo de procesamiento lógico humano**.

Una instrucción de alto nivel puede corresponderse con **varias instrucciones máquina**.



Clasificación según el método de ejecución



Compilados: El código fuente se traduce antes de ejecutarse a lenguaje máquina mediante un compilador. Generan un programa ejecutable independiente.

- Ventaja: más rápidos y eficientes en tiempo de ejecución.

Ejemplos: C, C++, Rust, Zig

JRE: MAQUINA VIRTUAL SOLO PARA EJECUTAR
PROGRAMAR APLICACIONES ES JDK

Interpretados: El código fuente se ejecuta línea a línea gracias a un intérprete. No generan ejecutable independiente: se necesita el intérprete en cada ejecución.

- Ventaja: facilidad de prueba y depuración, más flexibles.

Ejemplos: Python, JavaScript.

Híbridos: Combinan ambas ideas: primero compilan a un código intermedio, luego, en cada ejecución lo interpretan o ejecutan en una máquina virtual.

no es tan lento como interpretado ni rapido
como compilado .

Ejemplos: Java, C#.





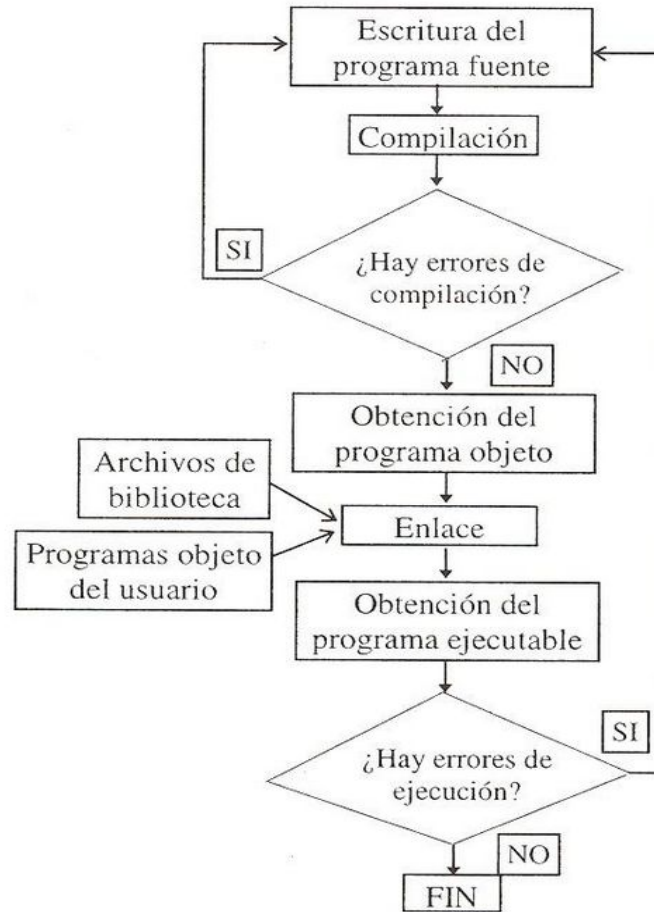
Es el programa que se encarga de traducir un programa fuente (aquel que está escrito en un lenguaje de alto nivel) a un **programa objeto** (semánticamente equivaliendo) escrito en lenguaje de bajo nivel.

El compilador **informa al usuario de los posibles errores** en lo programa fuente. Lo programa objeto solo se creará si no hay errores en el código fuente.

El programa objeto **se almacenará en un fichero que se podrá ejecutar sin necesidad de volverlo a traducir** (excepto si se han hecho cambios).

Un ejemplo es el compilador del lenguaje C. Lee el código fuente escrito en C, lo traduce y genera un programa ejecutable.

Proceso de compilación



Intérprete



Es un programa que **lee el código fuente** y va traduciendo y **ejecutando directamente sentencia a sentencia**.

Capta una sentencia del código fuente, la analiza y la interpreta y lo ejecuta directamente. **No se crea ningún fichero objeto.**

La principal diferencia entre un compilador y un intérprete, es que el compilador genera el ejecutable una vez y puede ser ejecuta muchas veces, mientras que un intérprete interpreta el código fuente cada vez que se quiera ejecutar.

Un **ejemplo de intérprete** es un **navegador web**. Se descarga una página web que puede contener el código fuente de un script de Javascript. Lo interpreta y lo ejecuta. El código fuente del script se descargará cada vez que se acceda a la misma página, se volverá a interpretar y a ejecutar.



Clasificación según el paradigma de programación



Imperativos: se enfocan en instrucciones secuenciales (ej.: C, Pascal).

Declarativos: describen qué debe hacerse y no cómo (ej.: SQL, Prolog).

Orientados a objetos: basados en clases y objetos (ej.: Java, C++).

Funcionales: basados en funciones matemáticas (ej.: Haskell, Lisp).

Multiparadigma: permiten combinar enfoques (ej.: Python, Scala, Rust).



El proceso de desarrollo



El proceso de desarrollo pasa por diferentes etapas.

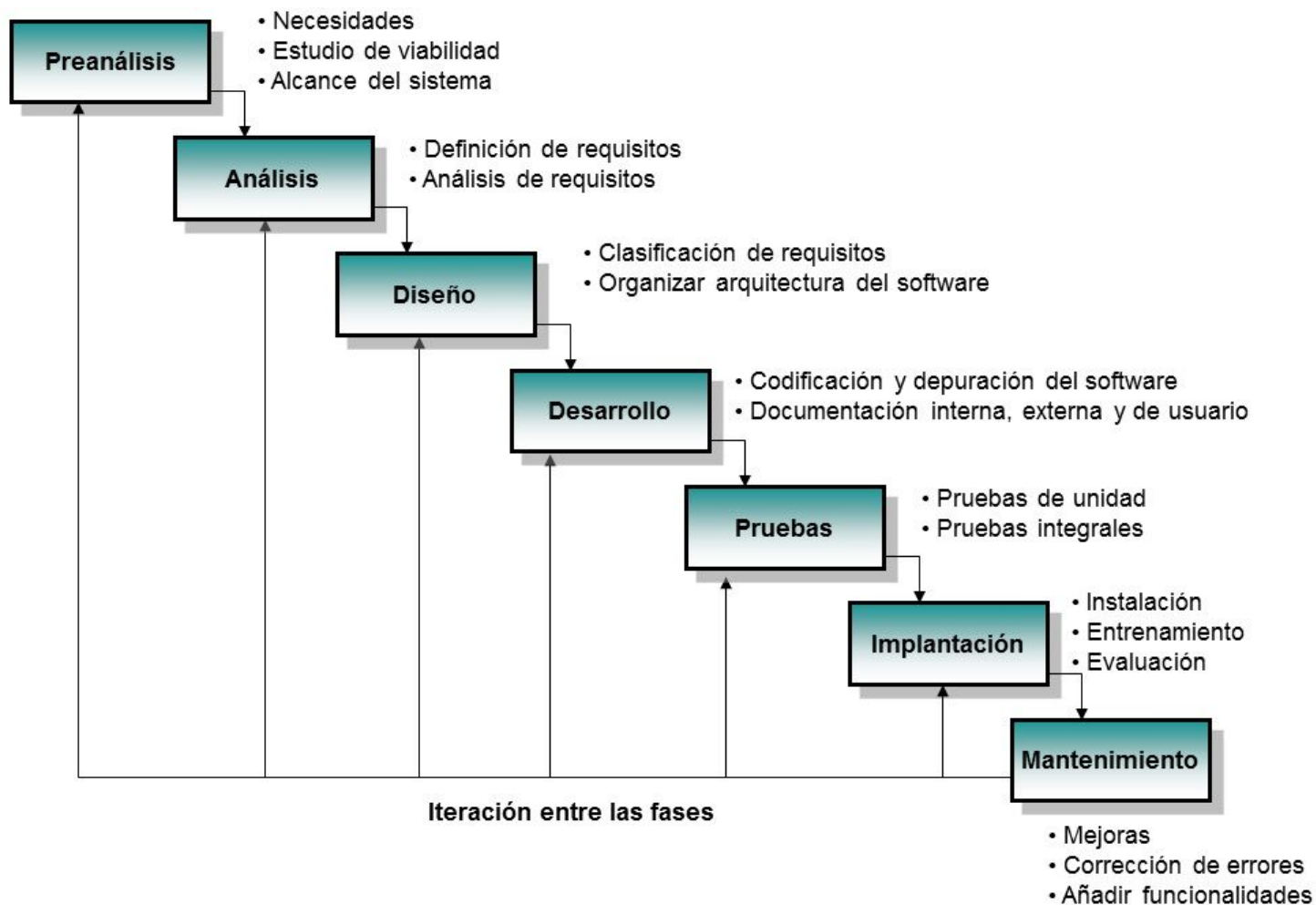
¿Qué etapas son?

- DESDE la necesidad de crear un SW
- HASTA que está listo para ser utilizado por el usuario final

Una buena definición de desarrollo de SW podría ser "proceso desde que se piensa la idea hasta que lo programado es implementado y utilizado por el usuario".



Ciclo de vida del software



Ciclo de vida del software

No todos los programas tienen ni hay que seguir fielmente las mismas etapas, pero es un buen punto de partida.

Existen más modelos de Ciclos de Vida del SW (iterativo, en espiral, basado en prototipos, en pruebas, etc). El que estudiamos es el modelo en cascada.



No hay que ceñirse a un modelo, se pueden mezclar.

>_



¿Por qué crees que existe tanto fracaso en el desarrollo de software?

Etapa de análisis

Importante comunicación **bilateral**.

No siempre se conoce el **contexto** o el **ámbito de aplicación**

El documento que se crea en esta etapa es el FSD (Functional Specification Document) o **especificación de requisitos**.

Muy importante tener claro **lo que quiere el cliente** o lo que se quiere desarrollar.

Suele hacerla un **analista** o un desarrollador con experiencia.

Es una de las etapas más importante de todas



Etapa de diseño



En base a los requisitos funcionales, **se determina cómo** se va a resolver el problema.

Se definen las **estructuras de datos**, **arquitectura SW**, **interfaz** de usuario, el **lenguaje de programación**, la **BBDD**.

Todas las **decisiones técnicas** se toman en esta etapa.

Todo queda reflejado en el TSD (Technical Specifications Document) que será utilizado por los programadores en la siguiente etapa.

Esta fase la suelen hacer los **analistas-programadores** o programadores con experiencia. También suelen estar presentes los arquitectos de HW.

Etapa muy importante.



Etapa de codificación



Se implementa todo lo que viene especificado en el documento de requisitos técnicos que se ha creado en base al análisis.

Cuando más detallado y exhaustivo sean las fases anteriores, más sencilla la tarea de codificación.

En esta etapa ya se genera documentación en el código.



>_



>_



Contacta con cliente.



Roles que intervienen en el desarrollo



Arquitecto de software (Tech Leader, Software Arch)

Toma decisiones sobre aspectos técnicos del proyecto.

Tiene un amplio conocimiento de las tecnologías, frameworks, etc.

Decide y forma los recursos de un proyecto.



Suele ser persona con conocimientos del campo de trabajo del proyecto



Roles que intervienen en el desarrollo



Analista programador (Programmer Analyst)

A caballo entre el funcional y el equipo de desarrollo.

Entiende las necesidades que le indica el funcional y las escribe para que los desarrolladores las entiendan.

Suele ser un programador con experiencia.



Roles que intervienen en el desarrollo



Programador (Developer)

Es necesario que conozca las tecnologías en las que trabajan.

Codifica las tareas asignadas por los analistas en los documentos de especificación.



Tester

Es habitual que sea el propio programador.

Se encarga de ejecutar la pila de pruebas planificadas para los módulos programados.



